NYCU IEE ICLAB – 2026 Spring

Midterm Project

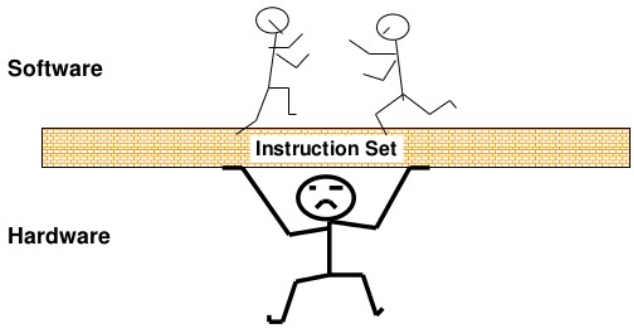
Design: Customized ISA Processor

Data Preparation

1. Extract test data from TA's directory:
   % tar xvf ~iclabta01/MP_2026_SPRING.tar

Design Description

A reduced instruction set computer (RISC) is a computer instruction set (AKA: the instruction set architecture (ISA)) which allows a computer's microprocessor to have fewer cycles per instruction (CPI) than a complex instruction set computer (CISC) [from wikipedia]. In this exercise, you are asked to design a 16-bits CPU with 8 registers and the CPU should be available for the required instruction set, which shows below (very similar to MIPS).



The following are the four formats for the core instruction set:

Type	-15-		- Format – (16-bits)			-0-
R	opcode (3-bits)	rs (3-bits)	rt (3-bits)	rd (3-bits)	func (3-bits)	?
Mult	opcode (3-bits)	rs (3-bits)	rt (3-bits)	rd (3-bits)	rl (3-bits)	?
I	opcode (3-bits)	rs (3-bits)	rt (3-bits)	immediate (7-bits)		
J	opcode (3-bits)	Address (13-bits)				

Register s (rs), Register t (rt), Register d (rd) and Register l (rl) represent the address of registers. Since the instruction takes 3 bits to store the address, it means we have 8 registers, from r0 to r7. Each register reserve 16 bits to store data, e.g. rs = 3'b000 means one of operands is "r0". rt = 3'b101 means one of operands is "r5", and so on.

Detailed instructions are shown below:

Function Name	Meaning	Type	Instruction Binary Encode
ADD	rd = rs + rt	R	000-rs-rt-rd-000?
SUB	rd = rs – rt	R	000-rs-rt-rd-001?
AND	rd = rs & rt (bit-wise)	R	000-rs-rt-rd-010?
OR	rd = rs | rt (bit-wise)	R	000-rs-rt-rd-011?
NAND	rd = ~(rs & rt) (bit-wise)	R	000-rs-rt-rd-100?
NOR	rd = ~(rs | rt) (bit-wise)	R	000-rs-rt-rd-101?
XOR	rd = rs ^ rt (bit-wise)	R	000-rs-rt-rd-110?
Set less than	if (rs < rt) rd = 1 else rd = 0	R	000-rs-rt-rd-111?
Mult	{rd, rl} = rs * rt	Mult	001-rs-rt-rd-rl?
ADDI	rt = rs + immediate (sign)	I	010-rs-rt-iiiiiii
SUBI	rt = rs – immediate (sign)	I	011-rs-rt-iiiiiii
Load	rt = DM[sign(rs+immediate)× 2 + offset]	I	100-rs-rt-iiiiiii
Store	DM[sign(rs+immediate)× 2 + offset] = rt	I	101-rs-rt-iiiiiii
Branch on equal	If (rs == rt) pc = pc + 1 + immediate(sign) else pc = pc + 1	I	110-rs-rt-iiiiiii
Jump	Next instruction address = address	J	111-address

*pc: program counter, meaning the next instruction. For R-type, Mult-type, and I-type instructions, pc should automatically +1 to get the next instruction.

*Each instruction has 16bits, which means the instruction address should +2 to get the next instruction.

*The offset of INST DRAM = 0x0000; the offset of DATA DRAM = 0x1000

*The first instruction address you acquire data should be "0x00000000".

```
@0000                                                    @00001000
a1 80 // I_inst: load 1000000010100001 rs = 0, rt = 1, imm =        33  65 00 // 101
@0002                                                    @00001002
3c 81 // I_inst: load 1000000100111100 rs = 0, rt = 2, imm =        60  23 85 //-31453
@0004                                                    @00001004
9d 81 // I_inst: load 1000000110011101 rs = 0, rt = 3, imm =        29  d2 00 // 210
```

Two instruction DRAM and one data DRAM are used in this lab. There are two cores in your design. One is used to decode instruction from instruction DRAM #1, and the other is used to decode instruction from instruction DRAM#2. Because there is only one data DRAM in this design, you need to make sure that the data you take from DRAM is newest one. You should be careful if you use the pipelining architecture, e.g. basic five-stage pipeline in a RISC machine, you must deal with data hazard problem.

use single cycle

Inputs & Output

I/O	Bit	Signal Name	Description
Input	1	clk	Positive edge trigger clock within clock period below 20.0ns
Input	1	rst_n	Asynchronous reset active low reset
Output	1	stall_1	Pull high when Core 1 is busy. It should be low for one cycle whenever you finished an instruction. TA will check the values of your registers of Core1 when your stall_1 is low. (refer to appendix to see how to check the register value in PATTERN)
Output	1	stall_2	Pull high when Core 2 is busy. It should be low for one cycle whenever you finished an instruction. TA will check the values of your registers of Core2 when your stall_2 is low. (refer to appendix to see how to check the register value in PATTERN)

- All inputs will be changed at clk ***negative*** edge.
- There is ***only 1 reset*** before the first pattern, thus, your design must be able to reset automatically.
- The test pattern will check the value in all registers of two cores at clock ***negative*** edge if stall is low.
- The test pattern will check the value in data DRAM every 50 instructions at clock negative edge if the last stall signal (stall_1 and stall_2) is low.
- All the registers of two cores should be zero after the reset signal is asserted.
- All the stall signals should be logic-high after the reset signal is asserted.
- The value in all registers should be **unchanged** when stall signal is low.

AXI 4 Interface (Connected with DRAM in Pattern)

In this project, the signal naming rule for AXI4 related signals is: AXI4 signal name (lower case) + _m_inf (suffix).

Write Address Channel

Signal	Source	Description
AWID[3:0]	Master	Write address ID. This signal is the identification tag for the write address group of signals. (In this project, we only use this to recognize master, reordering method is not supported)
AWADDR[31:0]	Master	Write address. The write address bus gives the address of the first transfer in a write burst transaction. The associated control signals are used to determine the addresses of the remaining transfers in the burst.
AWLEN[6:0]	Master	Burst length. The burst length gives the exact number of transfers in a burst. This information determines the number of data transfers associated with the address.
AWSIZE[2:0]	Master	Burst size. This signal indicates the size of each transfer in the burst. (In this project, we only support 3'b001 which is 2 Bytes (matched with Data Bus-width) in each transfer)
AWBURST[1:0]	Master	Burst type. The burst type, coupled with the size information, details how the address for each transfer within the burst is calculated. (only INCR(2'b01) is supported in this Project)
AWVALID	Master	Write address valid. This signal indicates that valid write address and control information are available: 1 = address and control information available 0 = address and control information not available. The address and control information remain stable until the address acknowledge signal, **AWREADY**, goes HIGH.
AWREADY	Slave	Write address ready. This signal indicates that the slave is ready to accept an address and associated control signals: 1 = slave ready 0 = slave not ready.

Write Data Channel

Signal	Source	Description
WDATA[15:0]	Master	Write data. The write data bus can be 8, 16, 32, 64, 128, 256, 512, or 1024 bits wide. (In this project, we only support 16 bit data width: WDATA[15:0])
WLAST	Master	Write last. This signal indicates the last transfer in a write burst.
WVALID	Master	Write valid. This signal indicates that valid write data and strobes are available: 1 = write data and strobes available 0 = write data and strobes not available.
WREADY	Slave	Write ready. This signal indicates that the slave can accept the write data: 1 = slave ready 0 = slave not ready.

Write Response Channel

Signal	Source	Description
BID[3:0]	Slave	Response ID. The identification tag of the write response. The BID value must match the AWID value of the write transaction to which the slave is responding.
BRESP[1:0]	Slave	Write response. This signal indicates the status of the write transaction. The allowable responses are OKAY, EXOKAY, SLVERR, and DECERR. (only OKAY(2'b00) is supported in this Project)
BVALID	Slave	Write response valid. This signal indicates that a valid write response is available: 1 = write response available 0 = write response not available.
BREADY	Master	Response ready. This signal indicates that the master can accept the response information. 1 = master ready 0 = master not ready.

only read operation to INST DRAM

Read Address Channel

Signal	Source	Description
ARID[3:0]	Master	Read address ID. This signal is the identification tag for the read address group of signals. (In this project, we only use this to recognize master, reordering method is not supported)
ARADDR[31:0]	Master	Read address. The read address bus gives the initial address of a read burst transaction.
ARLEN[6:0]	Master	Burst length. The burst length gives the exact number of transfers in a burst. This information determines the number of data transfers associated with the address.
ARSIZE[2:0]	Master	Burst size. This signal indicates the size of each transfer in the burst. (In this project, we only support 3'b001 which is 2 Bytes (matched with Data Bus-width) in each transfer)
ARBURST[1:0]	Master	Burst type. The burst type, coupled with the size information, details how the address for each transfer within the burst is calculated. (only INCR(2'b01) is supported in this Project)
ARVALID	Master	Read address valid. This signal indicates, when HIGH, that the read address and control information is valid and will remain stable until the address acknowledge signal, **ARREADY**, is high. 1 = address and control information valid 0 = address and control information not valid.
ARREADY	Slave	Read address ready. This signal indicates that the slave is ready to accept an address and associated control signals: 1 = slave ready 0 = slave not ready.

Read Data Channel

Signal	Source	Description
RID[3:0]	Slave	Read ID tag. This signal is the ID tag of the read data group of signals. The RID value is generated by the slave and must match the ARID value of the read transaction to which it is responding.
RDATA[15:0]	Slave	Read data. The read data bus can be 8, 16, 32, 64, 128, 256, 512, or 1024 bits wide. (In this project, we only support 16 bit data width: RDATA[15:0])
RRESP[1:0]	Slave	Read response. This signal indicates the status of the read transfer. The allowable responses are OKAY, EXOKAY, SLVERR, and DECERR. (only OKAY(2'b00) is supported in this Project)
RLAST	Slave	Read last. This signal indicates the last transfer in a read burst.
RVALID	Slave	Read valid. This signal indicates that the required read data is available and the read transfer can complete: 1 = read data available 0 = read data not available.
RREADY	Master	Read ready. This signal indicates that the master can accept the read data and response information: 1= master ready 0 = master not ready.

Specifications

1. Top module name: **DCCPU** (top file name: **DCCPU.v**)
2. It is **asynchronous** reset and **active-low** architecture. **rst_n** would active once at the beginning. All registers of two cores should be zero after the reset signal asserts; all stall signals should be one after the reset signal asserts.
3. **All D-flip-flop (DFF) in this design must be reset after the reset signal asserts.**
4. All the data are in **signed** format in all registers.
5. The maximum clock period is set to **20 ns**, and **you can determine the clock period by yourself**.
6. The input delay and the output delay should be **half** of the clk period, if clock period is 10ns, the input delay and the output delay will be 5ns.
7. The stall signals cannot be continuous high for 2,000 cycles during functionality check, cannot be continuous high for 100,000 during performance check.
8. The output loading is set to **0.05**. The target library is **slow.db**
9. The synthesis result of data type cannot include any **LATCH** (in syn.log).
10. In this project, you should **modify the syn.tcl (e.g. link library for your memory.db) by yourself**. **compile_ultra** will be used to synthesize. Since

memories are used in this project, the syn.tcl in Lab05 may be a good reference one.

11. The memory.db file should be the WC library.
12. You **CANNOT PASS** the demo if there are **timing violation** messages in the gate level simulation WITHOUT notimingchecks option
13. No **ERROR** is allowed in every simulation/synthesis.
14. **You must use SRAM in your design or you wouldn't get any point in this project.**
15. The maximum area should be smaller than **4,000,000.**
16. **You cannot change the register name or type from the original file.**
17. PATTERN will not generate any overflow cases during calculation.
18. The program counter will not over access range of memory when the jump instruction asserts. So, you need to think how to write your pattern by yourself.
19. For achieving the data dependence in real case, the next load/store address will be constrained in certain range for data memory and the address for jump/branch instruction will also be constrained in certain range.
20. The latency between BVALID and BREADY, RREADY and RVALID should not be larger than **300** cycles.
21. DRAM must be declared in PATTERN, **do not declare it in your DESIGN.**
22. Don't use any wire/reg/submodule/parameter name called *error*, *latch*, *congratulation* or *fail* otherwise you will fail the lab. Note: * means any char in front of or behind the word. e.g: error_note is forbidden.
23. Don't write Chinese comments or other language comments in the file you turned in.
24. Designware IPs are forbidden in this project.
25. Any form of display or printing information in verilog design is forbidden. You may use this methodology during debugging, but the file you turn in should not contain any coding that is not synthesizable.

Grading Policy

- RTL, Synthesis & Gate Level Simulation Correctness: 60%
  You can only get this score and pass the demo with correct RTL, synthesis, gate-level simulation (including functionality check and performance evaluation).
- Performance: 30%
  1. **Performance: Total Cycle x Clock Period x Area^2**
  2. You can only get performance score when you pass the demo.
  3. The ranking scores for the first demo and the second demo are separate.

TA will demo your design with 2 different patterns, one is for functionality check, the other is for performance evaluation.

For functionality check, the DRAM latency is in between 1 to 20, not fixed.

For performance evaluation, the DRAM latency is above 100 but below 500, not fixed.

- Pattern: 10%

  You need to submit your PATTERN.v. Your design must pass your own PATTERN using reasonable and correct methodologies.

- Bonus: 10%

  Report about your idea for PATTENR.v and DCCPU.v

  You can write the report in English or Taiwanese Mandarin. The total report must in the four pages.

  The file name of the report is iclabxxx.PDF and submit it to E3.

  TA reserves the right to withhold points based on the quality and substance of the content. Submission does not guarantee a score.

Note

1. Please submit your design file under the folder 09_SUBMIT before **12:00 noon**.

   Due Day : 1st Demo : April 29th 2nd Demo : May 1st

   In this lab, you can adjust your cycle time.

   After that, you should check the following files under

   09_SUBMIT/Midterm_Project_iclabXXX/

   RTL design : **DCCPU**_iclabXXX.v (XXX is your account no.)

   clock_cycle_iclabXXX.txt

   Memory file : 04_MEM_iclabXXX

   (with all your memory files .v & .db file)

   filelist_iclabXXX.f

   If you miss any files on the list, you will fail this lab.

   If any error occurs when restoring your design, you will FAIL the project.

   Then use the command like ./02_check to check the files are uploaded or not.

2. Template folders and reference commands:

   01_RTL/ (for RTL simulation) **./01_run_vcs_rtl**

   02_SYN/ (for Synthesis) **./01_run_dc_shell**

   Remember modify .tcl to fit your memory db name

   (Check the design which contains **Latch** and **Error** or not in **syn.log**)

   (Check the design's timing in /Report/***CPU.timing*** to see if the slack is **MET**)

   03_GATE / (Gate Level simulation) **./01_run_vcs_gste**

   You can key in **./09_clean_up** to clear all log files and dump files in each folder

   04_MEM/ (Memory location)

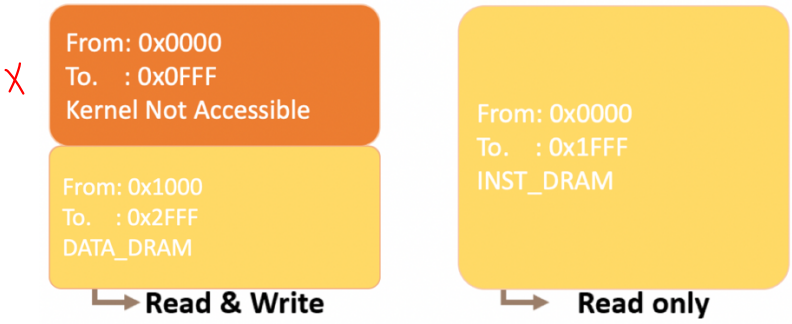
You should generate your memories and put the required files (.v and .db) here.

09_SUBMIT/ (Submit your files)

./00_tar ./01_submit ./02_check

Appendix

- TA will check the value in data memory **every 50 instructions and at the end of the simulation** when the last stall signal is low.
  The Instruction Counter (IC) starts counting from 0. Therefore, the 50th instruction refer to the instruction executed when the IC is 49.
- TA will check your registers of cores in PATTERN when the stall signals of the cores are low.
  Please make sure your registers can be read by PATTERN after synthesis, otherwise if TA's pattern can not access your register, you will fail your demo.
- Memory available range



- Store, load, branch and jump address
  The address value of the above instructions must be within the available address range for DRAM.
- The sequence of instructions between the two cores is strictly determined by the Instruction Counter (IC) for this project. For example, the 19th instruction of Core 1 is guaranteed to be executed before the 20th instruction of Core 2. Please pay close attention to the execution efficiency of both cores under this constraint.
- When both cores are at the same Instruction Counter (IC) value, the following rules apply to Load/Store operations:
  - **Store-before-Load Priority**: If Core 1 performs a Load instruction and Core 2 performs a Store instruction at the same IC, the Store operation must be completed first. For example, at IC = 20, if Core 2 stores to an address that Core 1 is loading from, Core 1 will receive the updated value from Core 2's store.
  - **Exclusive Store Access**: At any given IC, if one core performs a Store operation, the other core is strictly prohibited from performing any Store

operation, regardless of the target address.

- Instruction

  Ex. 000-0001-0010-0011-0 means r1 + r2 and saves the result into r3.

  The first instruction address you acquire data should be "0x00000000".

- Data Dependence Prediction

  For achieving the data dependence in real case, the next load/store address of cores will be constrained in certain range for one data DRAM and the address for jump/branch instruction will also be constrained in certain range. The range of jump instruction is from (current address – 256) to (current address + 256), e.g. if current pc is dec(1000), the next instruction pc will be from dec(872) to dec(1128).

  -128 , +128

